

Advanced Programming Project

B09 - AI-Assisted Multithreaded Mandelbrot Viewer

Leonardo Scoppitto №545615

March 2026

Contents

1	Introduction and Instructions	1
2	Design phase	2
3	Implementation	4
3.1	Chat based	4
3.2	First troubleshooting	8
3.3	AI agents, Opencode and Gemini CLI	9
3.4	Final backend review	11
3.4.1	Potential buffer overflows	11
3.4.2	Minor issues	12
4	Multithreaded performance evaluation	14
5	Conclusion	21
A	Appendix 1: Spec files	22
A.1	OpenGL Task	22
A.2	Webserver Task	23
B	Appendix 2: Lesson learned and concepts	24
B.1	Websockets from scratch	24
B.2	Webserver from scratch	24
B.3	The Mandelbrot Set: Definition and Algorithms	24
B.3.1	ThreadPool	25
B.3.2	SIMD and Parallel Computation	25
B.3.3	OpenGL	25
B.4	JavaScript 2D Canvas and Rendering	26

1 Introduction and Instructions

The project I chose is *B09 - AI-Assisted Multithreaded Mandelbrot Viewer with Zoom, Export, and Path Animation Goal*.

B09 - AI-Assisted Multithreaded Mandelbrot Viewer with Zoom, Export, and Path Animation Goal

Use AI to help design and implement a multithreaded Mandelbrot set viewer that supports interactive zooming, image export, and animation along a user-defined path into the fractal (e.g. zoom flight), leveraging concurrency to render efficiently.

Tasks

1. Design & Implement: Use AI to assist in implementing the Mandelbrot renderer (coloring, iteration control) with multithreaded rendering, an interactive UI for panning/zooming, image export (e.g. PNG), and an animation feature that interpolates a path of viewpoints and generates frames or a video-like sequence.
2. Test & Demonstrate: Demonstrate interactive usage (smooth zooming and navigation), validate correctness of the fractal rendering, measure/observe the impact of multithreading, and showcase at least one non-trivial animation path into the fractal.
3. AI Usage & Verification Report: Collect and submit the prompts you used with AI and write a short report explaining how you evaluated and corrected the AI-generated code (thread-safety, performance, correctness of the math, and quality of exported images/animations).

Requirements:

- Docker
- A modern browser

There are 4 docker compose files to run the backend in various configurations:

- `docker-compose.scalar.yml`: Uses scalar operations and can be run on any machine.
- `docker-compose.simd.yml`: Uses SIMD instructions and can be run on machines that support AVX/AVX2 instruction sets.
- `docker-compose.intel.yml`: The computation is offloaded to the integrated GPU. It requires an Intel iGPU.
- `docker-compose.nvidia.yml`: The computation is offloaded to the GPU. It requires an Nvidia GPU.

How to run it:

```
# To run the application
docker compose -f docker-compose.<version>.yml up --build

# To run the benchmark
cd backend
make BENCHMARKS="-DZOOM=0.5 -DTHREAD_PINNING" -j
make bench
```

When running the benchmark, the `ZOOM: float` level controls the load, while defining `THREAD_PINNING` makes the application threadpool pin each thread to a logical core of the CPU by setting the affinity.

2 Design phase

I already had an idea on how I wanted to implement this project, so here's a summary of the technical requirements that I imposed to the LLMs:

Requirement #1

The first requirement is **not to use third party libraries**. One of the benefits of using AI to speedup development is that, for simple and non critical use cases it is almost free to reinvent the wheel¹ and avoid the dependency hell that plagues modern software development².

Requirement #2

Use a **client-server architecture** to decouple the frontend and the backend. The other option was to implement a GUI application to manage both the computation and the visualization of the Mandelbrot, but the project would have become way more complex:

- Using a native UI framework would have required different codebases to target different OSes.
- Using a multi-platform framework (like Flutter or solutions like Tauri) could have been a valid solution, but, in my experience, the performance do not match a native solution.

Requirement #3

For the **frontend** I decided to implement a minimal web app so that with one codebase I could target any device with a modern browser. To limit the boilerplate at the minimum I opted for **plain HTML + JS + CSS** instead of a framework like React, which is suitable for more complex application with a complex state management, but in our case the application consists in a simple <canvas> element and a sidebar with a bunch of buttons.

Requirement #4

For the **backend** I wanted to be able to easily use **SIMD instructions** to speedup the rendering and to be able to implement **GPU rendering**, so the candidates were C, C++, Rust and Zig. In Table 1 I reported what I have considered when choosing the language. Ultimately, I opted for **C** because I worked with it professionally and because I prefer it for simple projects that require high performance and low level control.

¹Of course, core and critical libraries such as OpenSSL, complex protocol libraries or DB connectors are excluded from this argument.

²https://en.wikipedia.org/wiki/Dependency_hell.

Language	Pros	Cons
C	• Simple • Well-known by LLMs • Stable SIMD support via <code>immintrin.h</code> • Stable OpenGL library	• Manual memory management • Rudimental thread support
C++	• Well-known by LLMs • Support for promises and futures • Stable SIMD support via <code>immintrin.h</code> • Stable OpenGL library	• Good for big enterprise projects, but unnecessary complex for smaller projects
Rust	• Memory safe • Safe concurrency support via Arc wrappers and async multithreaded tokio runtime • Compiler hints help the LLMs (and agents) to iterate and fix the code faster	• SIMD support is still experimental • Harder and more complex to write and to review • OpenGL wrappers need unsafe blocks • I have limited experience with it
Zig	• Built-in advanced memory management via allocators • Better memory safety via <code>Optional</code> type and <code>defer/errdefer</code> mechanism • SIMD support out-of-the-box via <code>@Vector</code> • OpenGL support via C interop functionality	• It is still a relatively new language, so I do not expect LLMs to be as good as they could be with a more mainstream language • I have zero experience with this language

Table 1: Comparison between languages

3 Implementation

3.1 Chat based

To structure the project and set the foundation of the renderer and the visualizer, I used **Gemini 3 Pro** since it is free to use with the @unipi account.

Hello I have a implement this college project:

Multithreaded Mandelbrot Viewer with Zoom, Export, and Path Animation along a user-defined path into the fractal (e.g. zoom flight), leveraging concurrency to render efficiently.

I have a basic idea on how to architecture this:

- *Frontend: Basic HTML + JS + CSS to be able to view the mandelbrot. In a first iteration hw acceleration is not mandatory. We will discuss this later if there is time.*
- *Backend: I want to leverage SIMD (Intel AVX/AVX2) instructions to speedup the computation, other than using multithreading, so I was thinking about a C server that does the heavy-lifting of the computation.*

Now, the two components have to talk each other so I was thinking about websockets or better webRTC, but I can evaluate other options if you think are worth it.

An important rule: this has to be a ZERO dependency project, everything must be implemented from scratch, even the websocket/webrtc communication between the backend and the frontend

Please evaluate this proposal, highlight the strong and weak points and help me write down the implementation roadmap.

Gemini basically approved the tech stack and the constraints, suggesting to use websockets as they're easier to implement from scratch than WebRTC.

To visualize the madelbrot it suggested the canvas API (<canvas> tag) and it also suggested a basic communication protocol:

- The client sends [x_min,y_min,x_scale,y_scale,width,height,iterations] as a CSV string (I added the iteration count later as Gemini hardcoded it in the backend).
- The server responds with an ArrayBuffer containing the bitmap to be painted on the <canvas>.

Let's start with the math part. Please create a `render_simd` function to leverage AVX or AVX2 instruction sets conditionally for the correct target architecture (I have to test it on my laptop with AVX2 and on my server with AVX) and create a main function to benchmark the speedup when using single core, multicore and multicore + SIMD

Created successfully a benchmark program to test every combination (No SIMD single/multithreaded, SIMD single/multithreaded).

Go on implementing the WebSocket accept function, implementing the base64 encode and SHA1 checksum from scratch.

Created the two utility functions correctly and the library to upgrade an HTTP connection to a websocket, implementing also the send and receive functions.

Please finish the implementation creating the main server loop to accept requests from multiple clients and add a threadpool to render the mandelbrot.

It completed the backend application with the main loop so that I could start implementing the frontend.

At this point I started a new chat to prevent the LLM from hallucinating³ as it can happen with longer chats.

Before making the request I gave the LLM a bit of context about the project.

Let's start the frontend implementation. These are the rules:

- *Zero dependency: the frontend shouldn't require any external library*
- *Use pure HTML5 + JS + CSS, no frameworks.*
- *The style should be minimal and modern (dark theme Shadcn like)*
- *Remember the specification: Mandelbrot renderer with coloring, iteration control, an interactive UI for panning/zooming, image export (e.g. PNG), and an animation feature that interpolates a path of viewpoints and generates frames or a video-like sequence*
- *The protocol to be used over the websocket is:*

// The server expects:

```
"x_min,y_min,x_scale,y_scale,width,height,iterations"
scanf(payload, "%lf,%lf,%lf,%lf,%d,%d,%d", &x_min, &y_min,
&x_scale,
&y_scale, &width, &height, &iterations)
```

- *The data is returned as an ArrayBuffer so the receive part should look something like this:*

```
const socket = new WebSocket(`ws://${baseUrl}:8080`);
socket.binaryType = 'arraybuffer';

socket.onopen = () => {
  // Send initial coords
  socket.send("-2.0,-1.5,0.005,0.005,800,600");
};

socket.onmessage = (event) => {
  const arrayBuffer = event.data;
  const uint8Array = new Uint16Array(arrayBuffer);
  // Put this data into an HTML5 Canvas ImageData
};
```

It created the index.html, the style.css and the application logic script.js. The application opened in a browser seems to be fine, apart from minor UI inconsistencies that I manually fixed very quickly.

³It started deviating from the previous context using functions that didn't exist.

Can you create a makefile based on this structure to compile the project?

```
.
├── Makefile
└── src
    ├── include
    │   ├── mandelbrot.h
    │   ├── tpool.h
    │   ├── websocket.h
    │   └── websocket_utils.h
    ├── mandel_bench
    ├── mandelbrot.c
    ├── server.c
    ├── tpool.c
    ├── websocket.c
    └── websocket_utils.c
```

The Makefile was almost ok, except for a couple of compilation flags such as `-mfma` and `-mavx2`, which prevented the code from compiling on the older architecture of my server, so I changed them to `-march=native`, enabling all the optimization available on the machine that compiles the code, since the `render_simd` function automatically compiles to use the best available SIMD instruction set.

3.2 First troubleshooting

The code, **unexpectedly** compiled right away and the frontend tried to connect to the backend, but the backend crashed with a `segfault` a moment after the connection.

Inspecting the code manually revealed two problems:

1. The buffer is declared as an array of 4096 elements set to 0, but on line 10, the read function is allowed to overwrite the null terminator in the last position of the array. Since the buffer is later manipulated by `string.h` functions that expect the input string to be null terminated, I changed the buffer declaration to `char buffer[4097]`. However, this was not the bug that made the backend crash.

```
1 char buffer[4096] = {0};
2 // ...
3 client_fd =
4     accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
5 if (client_fd < 0)
6     continue;
7
8 printf("Client connected\n");
9 memset(buffer, 0, 4096);
10 read(client_fd, buffer, 4096);
11 printf("Received: %s\n", buffer);
12 if (perform_handshake(client_fd, buffer) != 0) {
13     close(client_fd);
14     continue;
15 }
16 printf("Handshake successful\n");
```

2. I manually isolated the function that caused the crash and it was the SHA1 implementation (see snippet below), where an underflow when computing `part_len`⁴ caused the `memcpy` try to copy terabytes of data. The LLM provided an update version that resolved the issue.

```
1 void sha1(const unsigned char *data, size_t len, unsigned char *hash) {
2     size_t i, j;
3     for (i = 0; i < len + 9; i += 64) {
4         unsigned char block[64] = {0};
5         size_t part_len = (i + 64 > len) ? len - i : 64;
6         memcpy(block, data + i, part_len);
7         // ...
8     }
9     // ...
10 }
```

At this point, the core of the project was working as expected and I could focus on details and new features.

⁴If `len < i`, then `part_len` underflows.

3.3 AI agents, Opencode and Gemini CLI

Once the project worked as it should, I took some time to review and audit the code:

- Check for redundancies and unused code
- Reduced the verbosity of some functions

Then, I containerized the application with docker to easily build and test it with few commands and have a stable reproducible test environment. So, I deployed:

- A container running the backend
- A container running Nginx to serve the frontend and proxy the requests to the backend

```
1  services:
2    backend:
3      build: ./backend
4      restart: unless-stopped
5      environment:
6        - MANDELBROT_THREADS=12
7        - MANDELBROT_SIMD=1 # Use AVX instructions set
8      expose:
9        - 8080
10
11   frontend:
12     image: nginx:latest
13     restart: unless-stopped
14     ports:
15       - 8080:80
16     volumes:
17       - ./nginx:/etc/nginx
```

Code 1: First docker-compose.yml

At this point, I had a reproducible environment to manually test the application, so the next step was to initialize a git repository to track the changes made by the agent and be able to rollback if needed. I create a new branch named `feature/opengl-support`, where to implement the support for rendering the mandelbrot using the GPU. I opted for OpenGL because it can target any GPU using the same shader. I also considered Vulkan, but a quick search revealed that it is way more complex than OpenGL.

Before issuing any task to the agent, I took some time review the content of the two main Gemini chats that produced the most of the code to create the foundations of the project documentation which has been summarized into `docs.md`.

First, I attempted to use Opencode wired to a local instance of `qwen3:8b` running via Ollama on one of my servers, but the GTX1070 with only 8Gb of VRAM was not enough to get a good development experience, resulting in an high waiting time before the first token and an output that was not on par with much larger models. Then I tried to connect my Gemini account to both OpenCode and Gemini CLI, to evaluate both tools⁵. Finally, I also tried OSS models like Kimi 2.5 or Qwen 3 by connecting an Nvidia Developer account to OpenCode⁶.

To guide the agent without having to write huge prompts, I created a new specification file (see Appendix A.1) that outlined the new feature to be developed, specifying the function signature, the new environment variables to be used to enable GPU rendering and the regression and acceptance

⁵I ended up preferring OpenCode as it allows to enqueue prompts while the agent is working to steer it. On the other hand, Gemini CLI evaluates enqueued prompts only when the agent stops, but it uses the tokens more efficiently, switching between high and low reasoning models when needed.

⁶The service is free but is best effort, so sometimes the models hang or are unreachable.

tests⁷ that the agent had to perform in order to decide whether the implementation was acceptable or not.

> Please read [@docs.md](#) to get a context of what we're doing and then read [@task.md](#) to implement the OpenGL support

Todos

- [✓] Add OpenGL headers to mandelbrot.h
- [✓] Implement render_opengl function in mandelbrot.c
- [✓] Add GPU environment variable support
- [✓] Update Makefile to link OpenGL libraries
- [✓] Configure docker-compose.yml for GPU passthrough

It successfully created the harness and the shader to make the function work, but the actual implementation used the scalar rendering even if it reported the successful implementation of GPU support:

```
1 void render_opengl(uint16_t *buffer, int width, int start_row, int end_row,
2                     int max_iterations, double x_min, double y_min,
3                     double x_scale, double y_scale, RenderMode mode) {
4
5     render_scalar(buffer, width, start_row, end_row,
6                  max_iterations, x_min, y_min, x_scale,
7                  y_scale);
8 }
```

With a bit of back and forth, I managed to direct the agent towards the solution, first by pointing out the issue and then iterating on the implementation and analyzing the logs together.

Now, I wanted to get rid of Nginx, as the project just needed a very basic webserver to serve files (i.e. responds to basic GET requests), so I created another `task.md` file (see Appendix A.2) with the steps needed to implement the functionality. Gemini managed to build it in one shot as it is a fairly easy task, also handling corner cases and preventing traversal attempts as I instructed it to do in the specification.

⁷The tests consist in:

- Running the backend benchmark to check that the already working functions still work
- Start the server and send some rendering requests using a python script to perform an integration test.

3.4 Final backend review

3.4.1 Potential buffer overflows

Analyzing the code I found that the `receive_frame` in `websocket.c` writes into the `out_payload` buffer without checking its length.

I fixed it passing the maximum length of the `out_payload` (parameter `size_t max_len`) and checking whether the incoming payload fits the buffer:

```
1 // In server.c the payload is allocated to the stack
2 // And initialized to 0.
3 char payload[WEBSK_PAYLOAD_LEN + 1] = {0};
4 while(receive_frame(client_fd, payload, WEBSK_PAYLOAD_LEN)) {
5     // process payload
6 }
7
8 int receive_frame(int client_fd, char *out_payload, size_t max_len) {
9     // Parse header and read length
10    if (payload_len >= max_len) {
11        // payload too large for buffer
12        return -1;
13    }
14    // Receive payload
15 }
```

I found a similar security issue also in the `perform_handshake` function, where the key was copied in a buffer without checking the length first. This issue has been addressed by explicitly checking the length of the incoming key:

```
1 // Original version
2 int perform_handshake(int client_fd, char *request_buffer) {
3     char *key_start = strstr(request_buffer, "Sec-WebSocket-Key: ");
4     if (!key_start)
5         return -1;
6
7     key_start += 19;
8     char *key_end = strstr(key_start, "\r\n");
9     if (!key_end)
10        return -1;
11    char key[64];
12    int key_len = key_end - key_start;
13    // key_len could be > 64
14    strncpy(key, key_start, key_len);
15    // ...
16 }
17
18 // Patched version
19 int perform_handshake(int client_fd, char *request_buffer) {
20     // ...
21    char key[KEY_LEN];
22    int key_len = key_end - key_start;
23    if (key_len >= KEY_LEN)
24        return -1;
25    // ...
26 }
```

3.4.2 Minor issues

The next issues are not as critical as the ones reported in the previous section, but have still been addressed.

SIMD:

In the `render_simd` function, the original function did not check whether the for loop iteration count is a multiple of 4⁸. The issue has been addressed by checking the bounds before the loop and implementing a scalar fallback in case there are leftover pixels to process:

```
1 void render_simd(/* ... */) {
2     // `width` is passed as a parameter
3     int vec_width = width & ~3; // Bitwise trick to round down to nearest multiple of 4.
4     // SIMD processing
5
6     // Scalar fallback. If the width is a multiple of 4
7     // this section does not run
8     for (int px = vec_width; px < width; px++) {
9         // Scalar operations
10    }
11 }
```

Webserver:

Despite having asked the LLM to check for path traversal in the webserver implementation, it just checked for the presence of `..` in the provided path. To make it more robust I changed that simple check with `realpath` making it more robust.

```
1 // Original version
2 static void serve_file(int client_fd, const char *root_path, const char *path) {
3     char full_path[MAX_PATH_LEN];
4     const char *target_path = path;
5     if (strstr(path, "..")) {
6         // Send 403 Forbidden
7         return;
8     }
9     // ...
10 }
11
12 // Patched version
13 static void serve_file(int client_fd, const char *root_path, const char *path) {
14     // ...
15     if (strcmp(path, "/") == 0) {
16         target_path = "/index.html";
17     }
18     snprintf(raw_path, sizeof(raw_path), "%s%s", root_path, target_path);
19     realpath(raw_path, resolved_path);
20     if (strncmp(resolved_path, root_path, strlen(root_path)) != 0) {
21         // Send 403 Forbidden
22         return;
23     }
24     // ...
25 }
26
```

⁸AVX/AVX2 instructions allow to process 4 pixels per iteration.

Best practices and error handling:

Finally, the last issue that I found is that the LLMs did not add any check after neither heap allocations nor system calls/pthread_* calls. While this could be fine for a simple application, I think it is a good habit to always check for errors (at least critical ones) and act accordingly:

- If the application is mission critical an error should be handled to ensure the least amount of damage and the best user experience possible.
- If the application is a simple demo or tool, it is still important to explicitly check for errors and then leave the application crash or exit.

```
1  #define CHECKALLOC(pointer) if(pointer == NULL) {fprintf(stderr, "Out Of Memory (file %s, line %d): %s\n", __FILE__, __LINE__);exit(EXIT_FAILURE);}
2  #define CHECKSC(call) \
3      if(call < 0){ \
4          fprintf(stderr, "Error (file %s, line %d): %s\n", __FILE__, __LINE__, strerror(errno)); \
5          exit(EXIT_FAILURE); \
6      }
7  #define CHECKPTHREAD(res) \
8      if(res != 0){ \
9          fprintf(stderr, "Error (file %s, line %d): %s\n", __FILE__, __LINE__, strerror(res)); \
10         exit(EXIT_FAILURE); \
11     }
12
13 // Examples
14 server->queue = malloc(sizeof(http_request_t) * queue_size);
15 CHECKALLOC(server->queue);
16 CHECKPTHREAD(pthread_mutex_lock(&server->mutex));
```

In this case, I decided to implement 3 macros to give useful information, such as where and why the error occurred, and then let the application exit with a non 0 return code.

4 Multithreaded performance evaluation

To evaluate the performance of the backend and the benefits of all optimizations, I relied on an internal benchmark that initializes the threadpool and executes multiple requests at different resolutions (outside the main cardioid) to get an estimation of the FPS that the backend is able to push. The results presented are the average of 5 consecutive runs.

To compute the speedup for benchmarks 1 to 3 I used as the baseline the SIMD single threaded execution for SIMD speedup and the scalar single threaded execution for scalar speedup⁹:

$$S_{\text{SCALAR}}(p) = \frac{\text{FPS}_{\text{SCALAR}}(p)}{\text{FPS}_{\text{SCALAR}}(1)} \quad S_{\text{SIMD}}(p) = \frac{\text{FPS}_{\text{SIMD}}(p)}{\text{FPS}_{\text{SIMD}}(1)}$$

Then, to evaluate the speedup introduced by using the vectorization, in benchmark 4, I computed the speedup using only the scalar single threaded execution as the baseline:

$$S_{\text{SIMD_SCALAR}}(p) = \frac{\text{FPS}_{\text{SIMD}}(p)}{\text{FPS}_{\text{SCALAR}}(1)}$$

Finally I measured the efficiency of the SIMD execution over the scalar execution, that should be ideally 4 as the SIMD implementation allows the processing of 4 pixels at a time:

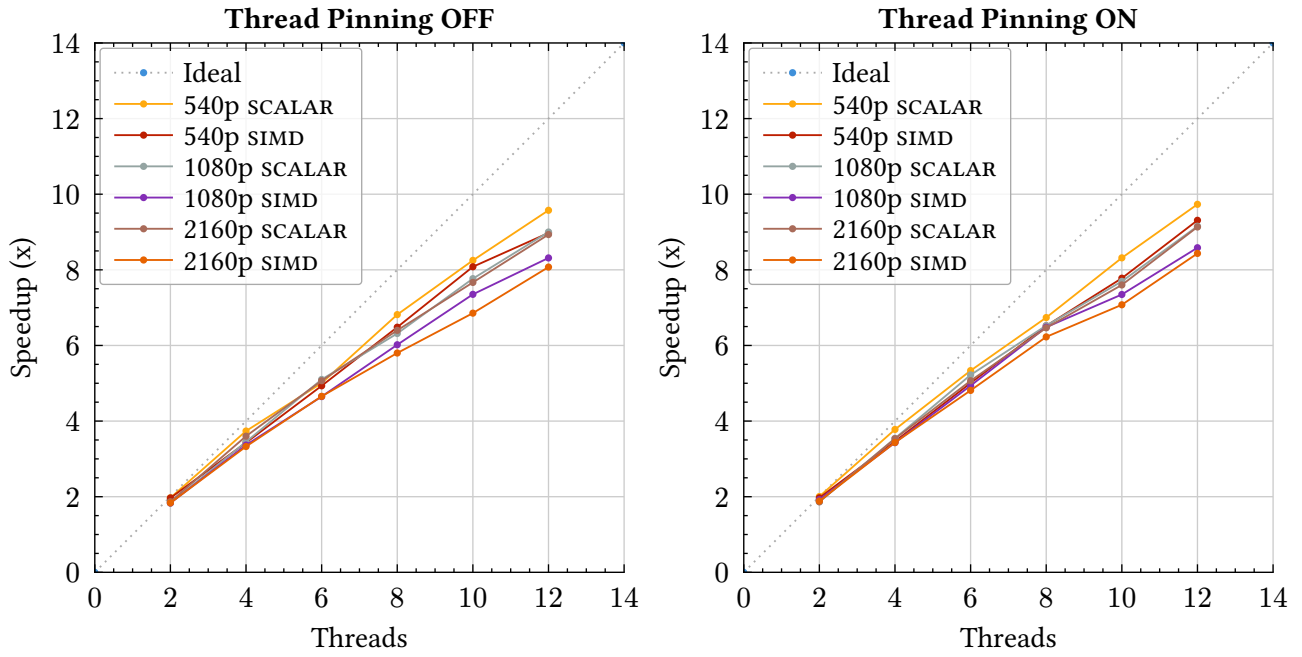
$$E_{\text{SIMD_SCALAR}}(p) = \frac{\text{FPS}_{\text{SIMD}}(p)}{\text{FPS}_{\text{SCALAR}}(p)}$$

Here it is what we can observe:

- The speedup in almost all executions is pretty much linear as the task is CPU bound. We can observe that for a higher ZOOM value the parallelization overhead is starting to sum up as the buffers become larger and the price of accessing the main memory starts to show.
- The workload doesn't benefit that much from setting the thread affinity (thread pinning) as the computation does not involve heavy reliance on the caches like when computing kernels and matrices multiplications.
- In benchmark 4, the ideal speedup of the SIMD execution over the scalar execution should be 4x, as it allows to process 4 pixels at a time. For brevity, I reported just the run with ZOOM=1, but all the executions show about the same results, where the SIMD execution achieves about a 3x speedup across the board as we can see in benchmark 5 where the SIMD efficiency is reported.
- Regarding the GPU performance, they are reported in benchmark 6, where we can see the huge difference between an integrated GPU and a dedicated one. Also, it interesting to note that the Intel GPU performs better at 1080p than at 540p.

⁹The definition of speedup is $S(p) = \frac{T(1)}{T(p)}$, but the FPS metric is already $\frac{1}{\text{Execution time}}$, so the speedup becomes the one reported.

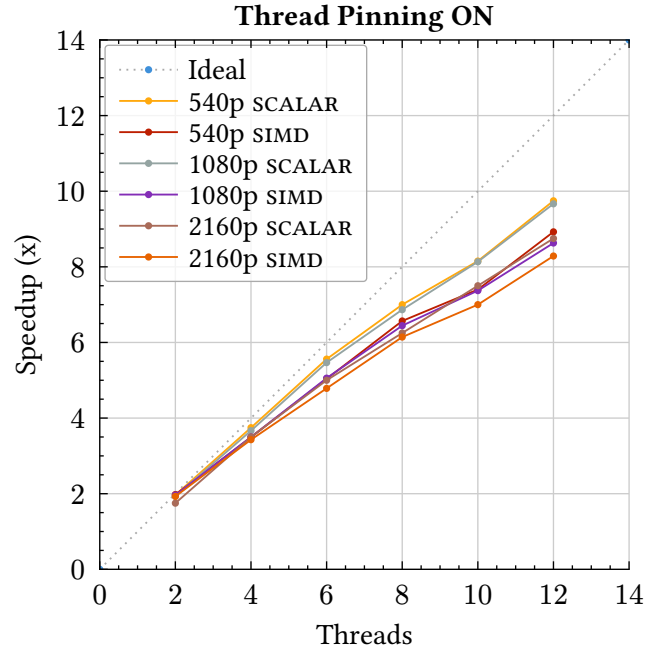
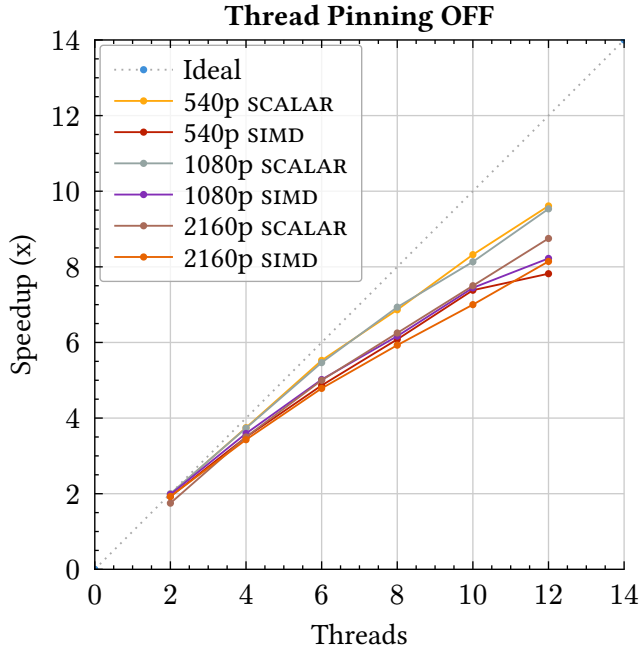
Benchmark 1: Run with ZOOM=0.5



Threads	540p SCALAR	540p SIMD	1080p SCALAR	1080p SIMD	2160p SCALAR	2160p SIMD
2	1.97x	1.97x	1.83x	1.83x	1.87x	1.84x
4	3.74x	3.43x	3.45x	3.37x	3.6x	3.33x
6	5x	4.93x	5.1x	4.65x	5.07x	4.65x
8	6.82x	6.48x	6.32x	6.02x	6.4x	5.8x
10	8.25x	8.08x	7.77x	7.35x	7.67x	6.85x
12	9.57x	8.98x	9x	8.31x	8.93x	8.07x

Threads (TP)	540p SCALAR	540p SIMD	1080p SCALAR	1080p SIMD	2160p SCALAR	2160p SIMD
2	2x	1.97x	1.86x	1.92x	1.87x	1.89x
4	3.78x	3.46x	3.54x	3.43x	3.53x	3.43x
6	5.33x	5.01x	5.22x	4.94x	5.07x	4.81x
8	6.74x	6.52x	6.53x	6.48x	6.47x	6.23x
10	8.32x	7.78x	7.69x	7.35x	7.6x	7.08x
12	9.73x	9.31x	9.15x	8.58x	9.13x	8.43x

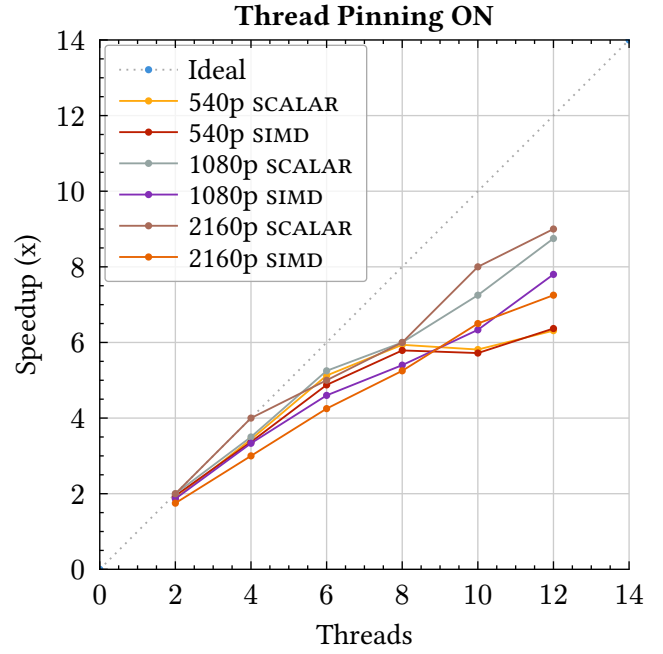
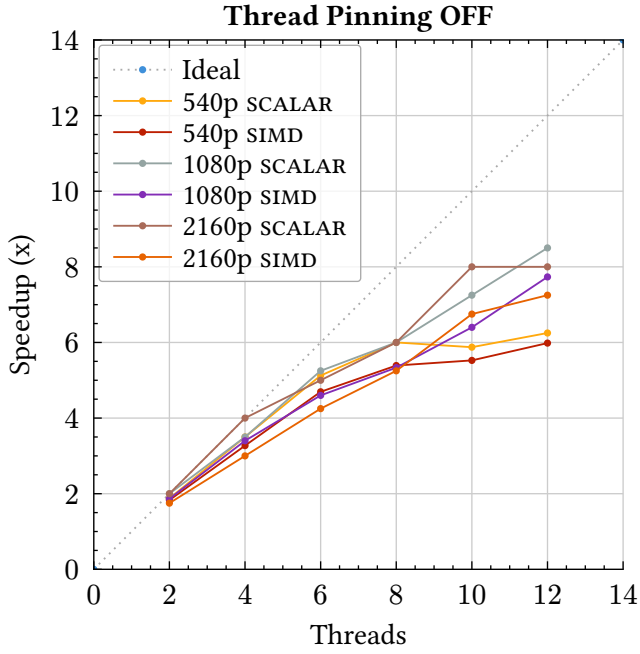
Benchmark 2: Run with ZOOM=1



Threads	540p SCALAR	540p SIMD	1080p SCALAR	1080p SIMD	2160p SCALAR	2160p SIMD
2	1.98x	1.93x	2x	1.98x	1.75x	1.93x
4	3.75x	3.5x	3.73x	3.59x	3.5x	3.43x
6	5.53x	4.86x	5.47x	5.02x	5x	4.79x
8	6.86x	6.08x	6.93x	6.17x	6.25x	5.93x
10	8.32x	7.38x	8.13x	7.44x	7.5x	7x
12	9.61x	7.82x	9.53x	8.22x	8.75x	8.14x

Threads (TP)	540p SCALAR	540p SIMD	1080p SCALAR	1080p SIMD	2160p SCALAR	2160p SIMD
2	1.97x	1.98x	1.93x	1.96x	1.75x	1.93x
4	3.75x	3.49x	3.67x	3.5x	3.5x	3.43x
6	5.56x	5.04x	5.47x	5.06x	5x	4.79x
8	7x	6.57x	6.87x	6.44x	6.25x	6.14x
10	8.15x	7.4x	8.13x	7.37x	7.5x	7x
12	9.75x	8.92x	9.67x	8.63x	8.75x	8.29x

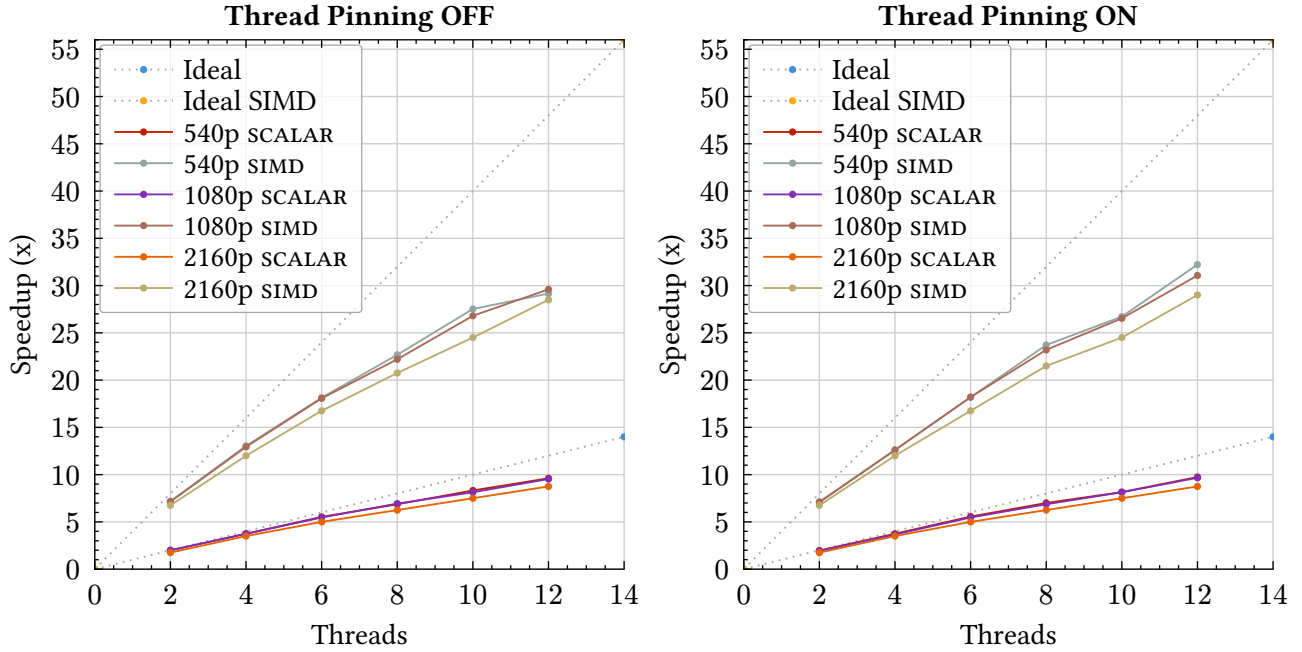
Benchmark 3: Run with ZOOM=5



Threads	540p SCALAR	540p SIMD	1080p SCALAR	1080p SIMD	2160p SCALAR	2160p SIMD
2	1.88x	1.83x	2x	1.87x	2x	1.75x
4	3.5x	3.27x	3.5x	3.4x	4x	3x
6	5.13x	4.69x	5.25x	4.6x	5x	4.25x
8	6x	5.39x	6x	5.33x	6x	5.25x
10	5.87x	5.53x	7.25x	6.4x	8x	6.75x
12	6.25x	5.98x	8.5x	7.73x	8x	7.25x

Threads (TP)	540p SCALAR	540p SIMD	1080p SCALAR	1080p SIMD	2160p SCALAR	2160p SIMD
2	1.88x	1.95x	2x	1.87x	2x	1.75x
4	3.44x	3.37x	3.5x	3.33x	4x	3x
6	5.13x	4.88x	5.25x	4.6x	5x	4.25x
8	5.94x	5.79x	6x	5.4x	6x	5.25x
10	5.81x	5.72x	7.25x	6.33x	8x	6.5x
12	6.31x	6.37x	8.75x	7.8x	9x	7.25x

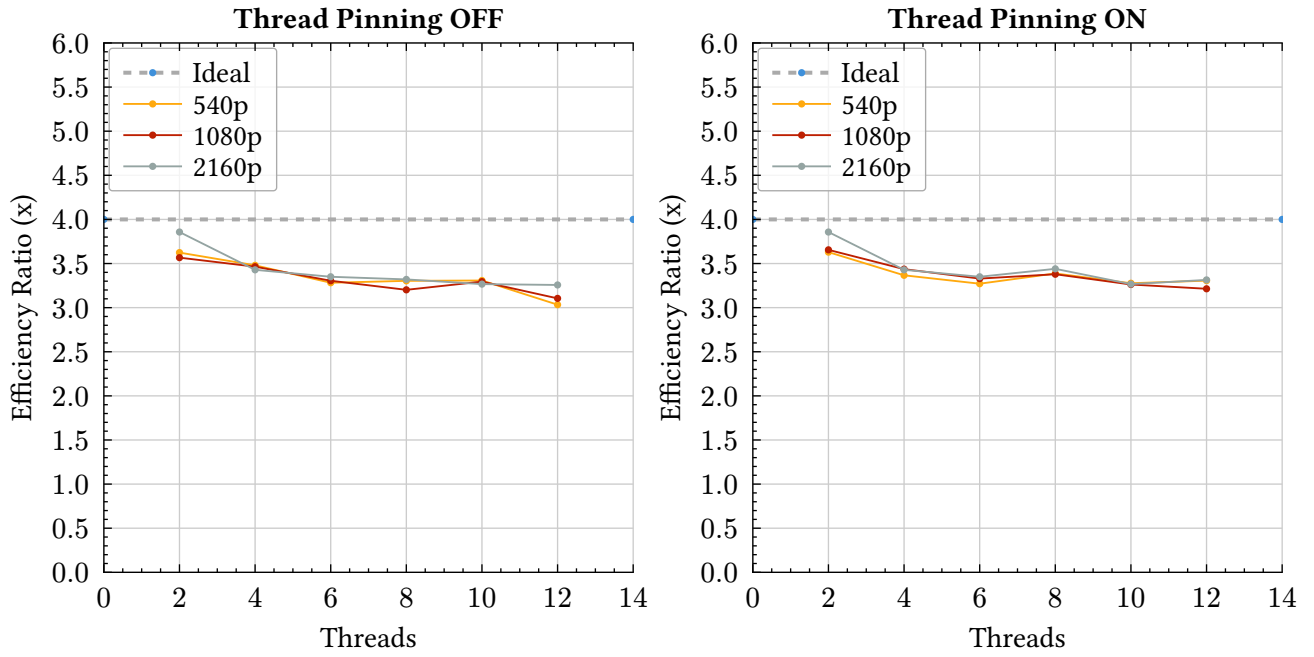
Benchmark 4: Run with ZOOM=1. SIMD speedup uses the single thread scalar execution as the baseline



Threads	540p SCALAR	540p SIMD	1080p SCALAR	1080p SIMD	2160p SCALAR	2160p SIMD
2	1.98x	7.19x	2x	7.13x	1.75x	6.75x
4	3.75x	13.03x	3.73x	12.93x	3.5x	12x
6	5.53x	18.14x	5.47x	18.07x	5x	16.75x
8	6.86x	22.68x	6.93x	22.2x	6.25x	20.75x
10	8.32x	27.53x	8.13x	26.8x	7.5x	24.5x
12	9.61x	29.15x	9.53x	29.6x	8.75x	28.5x

Threads (TP)	540p SCALAR	540p SIMD	1080p SCALAR	1080p SIMD	2160p SCALAR	2160p SIMD
2	1.97x	7.14x	1.93x	7.07x	1.75x	6.75x
4	3.75x	12.61x	3.67x	12.6x	3.5x	12x
6	5.56x	18.19x	5.47x	18.2x	5x	16.75x
8	7x	23.71x	6.87x	23.2x	6.25x	21.5x
10	8.15x	26.71x	8.13x	26.53x	7.5x	24.5x
12	9.75x	32.22x	9.67x	31.07x	8.75x	29x

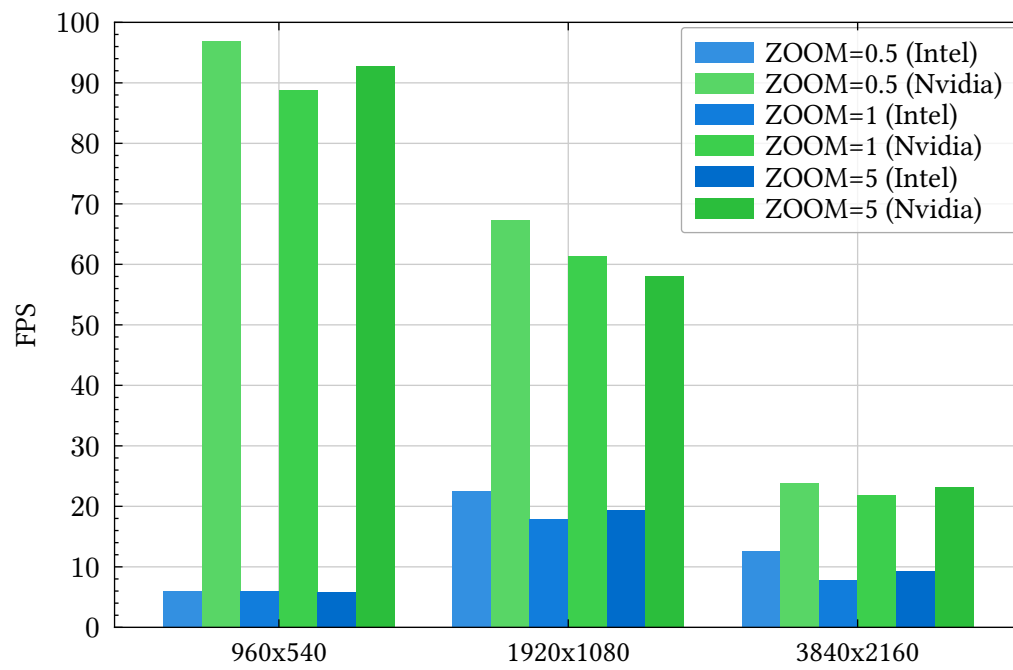
Benchmark 5: SIMD efficiency with ZOOM=1.



Threads	540p	1080p	2160p
2	3.62x	3.57x	3.86x
4	3.48x	3.46x	3.43x
6	3.28x	3.3x	3.35x
8	3.3x	3.2x	3.32x
10	3.31x	3.3x	3.27x
12	3.03x	3.1x	3.26x

Threads (TP)	540p	1080p	2160p
2	3.63x	3.66x	3.86x
4	3.37x	3.44x	3.43x
6	3.27x	3.33x	3.35x
8	3.39x	3.38x	3.44x
10	3.28x	3.26x	3.27x
12	3.31x	3.21x	3.31x

Benchmark 6: GPU



5 Conclusion

Before starting the project, I already knew a bit of all the technologies that I ended up using, but surely I wouldn't have been able to implement all these features in such a small amount of time. AI clearly helped me to deep dive into each aspect of the project where I wasn't so knowledgeable¹⁰ and also speeded up the implementation part, even when I could have been perfectly capable of writing the same code.

Regarding the development experience, a distinction between chatbots and agents has to be made:

- Going back and forth between the chat and the editor is useful in the researching phase as it is a more natural interface to interact with an LLM to brainstorm ideas, but it is not so good when you have to start implementing things seriously.
- Using a coding agent, given that you provide extended and well written requirements, is extremely powerful, as it can accomplish much more than a chatbot. Moreover, if it can use well structured test and have access to verbose compilation logs, it can work for longer trying to fix bugs and implement features one after the other without human interaction.

That said, while AI is undeniably one of the most powerful productivity tools when it comes to coding, a grain of salt is needed when using it:

- Both chatbots and agents **need** human supervision, diligent code hygiene practices and a good versioning system, as subtle bugs can slip into the code at any time and things can break unexpectedly after the wrong prompt. I think it's still crucial that each line produced must be audited and reviewed, even if solid tests are in place, making necessary to have constant review sessions, otherwise the amount of produced code can quickly become unmanageable. Some say that every line of code is both a cost and a liability, so we can say that AI creates liabilities at the speed of light.
- Even if the developer is still the owner of the blueprint and the requirements, navigating the codebase becomes harder than before, as the implementation constantly evolves at high speed, which combined with what said above, poses two scenarios:
 - The speed introduced by AI is bottlenecked by the review speed of humans
 - The code is not fully audited and humans lose track of the codebase

Given the increasing amount of bugs I'm experiencing using all kinds of software, sadly, I think the second scenario is the one's winning.

- In my opinion, while it democratizes knowledge and gives access to an infinite amount of resources, it makes the learning experience optional and less effective, as one could be tempted to get answers fast and go directly to the solutions without reasoning about things.

To conclude, I think AI is a revolutionary technology, but, as of now, I think it is not ready to be widely adopted in production environments, except as an enhanced Google Search or as an assistant to speed up non-critical tasks, especially for junior figures. On the other hand, I think that it shouldn't be used by younger students (high school and younger) if not supervised or in a controlled manner, to avoid degrading the learning opportunities.

¹⁰See Appendix 2 for a quick summary of what I learned implementing this project.

A Appendix 1: Spec files

A.1 OpenGL Task

```
1
2 # Specification
3
4 We want to leverage the computational power of the GPU to render the Mandelbrot more quickly:
5
6 - Headless Context: Uses EGL to create an OpenGL context without a windowing system (X11/
  Wayland), enabling the server to run in Docker or on headless servers.
7 - Compute Shader: The rendering logic is written in GLSL (`#version 430`).
8 - Precision Optimization: The shader automatically switches between `float` (FP32) and
  `double` (FP64). If the zoom level is shallow (`x_scale > 1e-7`), it uses `float` for
  significantly faster performance.
9 - Persistent Mapping: Uses `glMapBufferRange` with `GL_MAP_PERSISTENT_BIT` to map GPU memory
  directly to the CPU's address space, minimizing data transfer overhead.
10 - GPU Selection: Create a utility function to be able to set the GPU to be used.
11
12 The function signature has to be:
13
14 void render_opengl_frame(uint16_t *buffer, int width, int start_row,
15                          int end_row, int max_iterations, double x_min,
16                          double y_min, double x_scale, double y_scale);
17
18 # Todos
19
20 - [ ] Add OpenGL headers to mandelbrot.h
21 - [ ] Implement render_opengl function in mandelbrot.c
22 - [ ] Add GPU environment variable support
23 - [ ] Update Makefile to link OpenGL libraries
24 - [ ] Configure docker-compose.yml for GPU passthrough
```


A.2 Webserver Task

```
1  ## Next features
2
3  As of now the project is composed by a frontend and a backend. The frontend is served by an nginx
4  instance and consists in 3 static files:
5  - index.html
6  - script.js
7  - style.css
8
9  Now we want to:
10 - [ ] implement a webserver in our backend.
11 - [ ] make the backend serve both the mandelbrot generation and the webpages, being aware to
12   implement the basic security features such as avoid use after free, no buffer overflows and so
13   on.
14 - [ ] the amount of requests is small so the webserver should run in its own thread with its own
15   request queue.
16 - [ ] The page should be served from `/var/www` and the requests should not escape from that
17   path.
18
19 A note on the backend project structure:
20
21 .
22 └─ benchmark.log <- ignore it
23 └─ Dockerfile <- dockerfile without nvidia framework
24 └─ Dockerfile.nvidia <- dockerfile with nvidia framework
25 └─ frontend <- frontend directory
26   └─ index.html
27   └─ script.js
28   └─ style.css
29 └─ Makefile <- makefile for compilation
30 └─ plot_benchmark.py <- ignore it
31 └─ src <- C backend source
32   └─ include <- headers
33   └─ mandelbrot.c <- mandelbrot rendering
34   └─ server.c <- entrypoint and server implementation
35   └─ tpool.c <- mandelbrot threadpool implementation (for the webserver use another thread)
36   └─ webserver.c <- Implement webserver HERE
37   └─ websocket.c <- websocket handshake and send/recieve
38   └─ websocket_utils.c <- websocket utils functions
39   └─ test_one_frame.py
```

B Appendix 2: Lesson learned and concepts

B.1 Websockets from scratch

WebSockets provide a full-duplex communication channel over a single TCP connection. The protocol begins with an HTTP handshake:

1. The client sends an upgrade request containing a Sec-WebSocket-Key.
2. The server takes this key, appends a specific GUID string, computes the SHA-1 hash, and returns the base64-encoded result in the Sec-WebSocket-Accept header.
3. Once established, data is exchanged in frames. Each frame contains control bits (fin, opcode), a payload length (which can be extended), a masking key (for client-to-server messages), and the payload itself.

In this project, the implementation is handled in `src/websocket.c` and does not rely on external libraries:

- **Handshake:** The `perform_handshake` function searches for the Sec-WebSocket-Key header, concatenates it with the magic GUID 258EAF5-E914-47DA-95CA-C5AB0DC85B11, computes the SHA-1 hash, and Base64 encodes the result to complete the upgrade.
- **Frame Parsing:** `receive_frame` interprets the first two bytes to determine the opcode (checking for connection close 0x8) and payload length. It handles variable-length payloads (7-bit, 16-bit extended, or 64-bit extended) and unmask the client data using the 4-byte masking key (`buffer[i] ^ mask_key[i % 4]`).
- **Binary Responses:** The `send_binary_frame` function constructs valid WebSocket frames with the FIN bit set and opcode 0x2 (Binary), ensuring large image buffers are transmitted correctly by setting the appropriate extended payload length bytes in big-endian order.

B.2 Webserver from scratch

Since the webserver functionalities needed for the project were minimal, I decided to implement it from scratch, instead of using a solution like Nginx or Apache, making the app self contained.

The `src/webserver.c` implementation features:

- **Request Parsing:** A dedicated worker thread (`webserver_worker`) parses the raw HTTP request string using `sscanf` to extract the method and path.
- **Security:** The `serve_file` function enforces a root directory jail using `realpath`, returning 403 Forbidden if a traversal attempt is detected.
- **Concurrency:** A producer-consumer queue (`webserver_enqueue`) allows the main thread to offload file serving tasks to the webserver thread, preventing static file I/O from blocking the Mandelbrot rendering loop.

B.3 The Mandelbrot Set: Definition and Algorithms

The Mandelbrot set is defined as the set of complex numbers c for which the sequence $z_{n+1} = z_n^2 + c$ (starting with $z_0 = 0$ and $c = x^2 + iy^2$) does not diverge to infinity. Mathematically, if $|z_n| > 2$, the sequence is guaranteed to diverge. The “escape time” algorithm iterates this formula up to a maximum limit; the number of iterations reached determines the pixel’s color. So, the black pixels are the ones belonging to the set, while the color shows how “far” the point is from the set.

The algorithm is implemented in `src/mandelbrot.c`:

- **Scalar Loop:** The `render_scalar` function implements the classic escape time algorithm. It uses the optimization $x^2 + y^2 < 4.0$ to avoid expensive square root calculations.
- **Algebraic Simplification:** The update rule is expanded to $x_{\text{new}} = x^2 - y^2 + x_0$ and $y_{\text{new}} = 2xy + y_0$. By maintaining x^2 and y^2 as separate variables in the loop, the implementation minimizes the number of multiplication operations per iteration.

B.3.1 Threadpool

To use multi-core processors efficiently without the overhead of constantly creating and destroying threads, a threadpool was implemented. I decided not to use something like OpenMP APIs because I wanted to implement a threadpool from scratch in C.

Implemented in `src/tpool.c`:

- **Synchronization:** The `tpool_t` structure uses a `pthread_mutex_t` to protect the task queue and a `pthread_cond_t` to signal sleeping worker threads when new work is available.
- **Task Queue:** A circular buffer holds function pointers (`thread_task_t`). When the main loop needs to render an image, it decomposes the frame into horizontal strips and pushes these tasks to the pool via `tpool_add_work`.
- **Worker Loop:** Threads run an infinite loop (`thread_worker`), blocking on the condition variable until a task is available, executing it, and then returning to sleep, ensuring low latency and high CPU utilization.

B.3.2 SIMD and Parallel Computation

To speed up the mandelbrot computation I decided to employ the techniques learned attending the *Distributed systems: paradigms and models* course. Single Instruction, Multiple Data (SIMD) allows a processor to perform the same operation on multiple data points simultaneously on the same ALU. The project leverages AVX2 (Advanced Vector Extensions) to process four double-precision floating-point numbers at once. To maximize compatibility, the instruction sets supported are AVX and AVX2. The `render_simd` function in `src/mandelbrot.c` holds the SIMD implementation of the mandelbrot rendering:

- **Vectorized Math:** It uses intrinsic functions like `_mm256_mul_pd` and `_mm256_add_pd` to perform the complex number arithmetic on four pixels simultaneously (`_m256d`).
- **Conditional Masking:** Since different pixels diverge at different speeds, standard `if/break` logic cannot be used. Instead, `_mm256_cmp_pd` creates a comparison mask. The iteration count is updated using `_mm256_sub_epi64` only for those elements in the vector that satisfy the condition ($|z|^2 < 4$).
- **Packing:** The final 64-bit integer counters are packed down to 16-bit integers using permutation (`_mm256_permutevar8x32_epi32`) and packing (`_mm_packus_epi32`) instructions for efficient memory storage.

B.3.3 OpenGL

Finally, I wanted to dive in GPU programming as I never tried to run any algorithm on a graphic accelerator. To implement this part I instructed the agent to use OpenGL since I knew it supports multiple GPUs vendors, meaning that with one algorithm I could render the mandelbrot both on NVIDIA and Intel GPUs (the ones I have in my laptop). OpenGL is a cross-language, cross-platform API for rendering vector graphics. While primarily designed for visual output, it can be also used for general-purpose computing.

The system abstracts the GPU hardware in `src/mandelbrot.c`:

- **Compute Shaders:** A GLSL compute shader (`#version 430 core`) handles the heavy lifting. It runs one thread per pixel (`gl_GlobalInvocationID`) and implements the Mandelbrot logic, including a geometric optimization that skips calculations for points inside the main cardioid.
- **Persistent Mapping:** To minimize driver overhead, the project uses `glMapBufferRange` with the `GL_MAP_PERSISTENT_BIT`. This maps GPU memory directly into the CPU's address space (`gl_mapped_buffer`), allowing the application to read back render results without explicit buffer copy commands.

- **Vendor Agnostic:** The `init_opengl_for_vendor` function dynamically scans available EGL devices to find and initialize the correct GPU (Intel or NVIDIA), ensuring the application runs on the desired hardware backend.

B.4 JavaScript 2D Canvas and Rendering

Finally, regarding the frontend I decided not to use a framework such as React, but to develop the webapp using only HTML, CSS and JavaScript so that I could learn the basics of web development, which are hidden by the abstractions provided by modern frameworks. Also, I did not opt for a native GUI because it cannot be easily distributed across different operating systems and architectures, while a browser is available on almost all platforms. Also, this way, the rendering and the visualization can happen on different machines.

The webapp is very simple and features a sidebar with all the controls and a `<canvas>` element which provides a drawing surface where to paint the Mandelbrot. For high-performance rendering, this project uses the `CanvasRenderingContext2D` API, which can leverage the GPU acceleration to paint and render the image.

The `script.js` handles the application logic:

- **Binary Protocol:** The client sends coordinates as a string and receives a raw binary `ArrayBuffer` containing 16-bit iteration counts.
- **Client-Side Coloring:** To allow instant theme switching without re-requesting data, the mapping from iteration count to RGBA color happens in the browser. A `Uint32Array` palette is precomputed.
- **Buffer Management:** The received data is wrapped in a `Uint16Array`. The script iterates through this array, looks up the color in the palette, and writes the resulting 32-bit pixel value into an `ImageData` buffer's `Uint32Array` view.
- **Drawing:** The final image is pushed to the GPU-accelerated canvas using `ctx.putImageData` (or `createImageBitmap` for scaling), providing a smooth experience.